

Codificación en Lenguaje Máquina y Simulación del Ciclo de Instrucción de una Arquitectura CPU Imaginaria

David Muñoz Barrón

Ingeniería en Sistemas, Grupo 3C — Arquitectura de Computadoras

Universidad Autónoma de Aguascalientes

Abstract—Este trabajo documenta la resolución de un ejercicio en clase sobre el ciclo de instrucción de un procesador. A partir de una arquitectura de CPU imaginaria propuesta en clase —cinco códigos de operación, un formato de instrucción de 9 bits y un espacio de direcciones de memoria de 3 bits— se tradujo a lenguaje máquina un algoritmo aritmético de cuatro asignaciones, y se simuló etapa por etapa (Fetch, Decode, Execute, Memory/Write Back) la ejecución de cada instrucción codificada. El estado final de las memorias obtenido en la simulación coincide con el resultado esperado del algoritmo original, evidenciando de forma práctica cómo una operación de alto nivel se descompone en las fases del ciclo de instrucción de un procesador.

I. INTRODUCCIÓN

Para ejecutar un programa, un procesador repite de manera continua una secuencia de etapas conocida como ciclo de instrucción: Fetch (obtener la siguiente instrucción desde memoria), Decode (interpretarla y determinar la operación a realizar), Execute (realizar la operación indicada) y Memory/Write Back (acceder a memoria y almacenar el resultado). Estas etapas pueden además dividirse y solaparse mediante técnicas como el pipeline, la predicción de saltos o la ejecución especulativa, pero su forma secuencial básica es la unidad mínima de trabajo de cualquier CPU.

Dentro de la Unidad I del curso, y en relación directa con la distinción entre la ISA (Instruction Set Architecture) de un procesador y su implementación física, el profesor propuso un ejercicio con una arquitectura de CPU imaginaria: dado un conjunto reducido de códigos de operación y un formato de instrucción fijo, se pidió traducir un algoritmo sencillo a lenguaje máquina y, después, simular la ejecución de cada instrucción codificada mostrando su paso por las cuatro etapas del ciclo. El objetivo de este reporte es documentar dicho procedimiento y sus resultados.

II. ARQUITECTURA PROPUESTA Y METODOLOGÍA

A. Conjunto de instrucciones (ISA)

La arquitectura imaginaria define cinco operaciones, cada una identificada por un código de 3 bits:

Operación	Código
Suma	000
Resta	001
Inc. +1	010
Dec. -1	011

Mover a Memoria	100
-----------------	-----

TABLA I. Códigos de operación de la arquitectura imaginaria.

B. Formato de instrucción

Cada instrucción ocupa 9 bits, divididos en tres campos de 3 bits: código de operación, Operando A y Operando B. En las operaciones aritméticas (Suma, Resta, Inc., Dec.) el resultado se almacena siempre en la Memoria 1, que funciona como un acumulador fijo; por esa razón, llevar un resultado a cualquier otra dirección requiere una instrucción adicional de Mover a Memoria.

C. Mapeo de direcciones de memoria

Las direcciones de memoria también se codifican en 3 bits. La captura proporcionada en clase confirma las primeras tres direcciones; las direcciones de Memoria 4 y 5 se asumieron siguiendo el mismo patrón binario correlativo:

Nombre	Dirección
Memoria 1	001
Memoria 2	010
Memoria 3	011
Memoria 4*	100
Memoria 5*	101

TABLA II. Mapeo de direcciones de memoria (*Memoria 4 y 5 asumidas por patrón correlativo).

D. Algoritmo a codificar

```
Memoria 2 = 2 + 2
Memoria 3 = 5 + 1
Memoria 4 = Memoria 2
Memoria 5 = 2 - 1
```

E. Procedimiento de traducción

Se identificó que “5 + 1” y “2 - 1” corresponden a incrementos y decrementos unitarios, por lo que se codificaron con Inc. +1 y Dec. -1 respectivamente, en lugar de las operaciones genéricas Suma/Resta. Como todo resultado aritmético cae en Memoria 1, cada línea que asigna a otra dirección se tradujo como dos instrucciones: el cálculo, seguido de un Mover a Memoria desde Memoria 1 hacia el destino. La línea “Memoria 4 = Memoria 2” no involucra aritmética, por lo que se tradujo como un único Mover a Memoria directo entre ambas direcciones.

III. DESARROLLO Y RESULTADOS

A. Traducción a lenguaje máquina

#	Instrucción	OP	A	B	BIN (9 bits)
I1	Suma (2, 2) → M1	000	010	010	000010010
I2	Mover M1→M2	100	001	010	100001010
I3	Inc.+1 (5) → M1	010	101	000	010101000
I4	Mover M1→M3	100	001	011	100001011
I5	Mover M2→M4	100	010	100	100010100
I6	Dec.-1 (2) → M1	011	010	000	011010000
I7	Mover M1→M5	100	001	101	100001101

TABLA III. Instrucciones codificadas. Las cuatro líneas del algoritmo original se expanden a siete instrucciones de máquina, consecuencia directa de que el conjunto de instrucciones no permite especificar un destino distinto de Memoria 1 en las operaciones aritméticas.

B. Simulación del ciclo de instrucción

Para cada instrucción codificada se muestra su paso por las cuatro etapas del ciclo y el efecto observable sobre la memoria:

#	Fetch	Decode	Execute	Memory / WB
I1	000010010	Suma, A=2, B=2	ALU: 2+2=4	escribe M1 (4)
I2	100001010	Mover, orig.=M1, dest.=M2	lee M1 (4)	escribe M2 (4)
I3	010101000	Inc.+1, A=5	ALU: 5+1=6	escribe M1 (6)
I4	100001011	Mover, orig.=M1, dest.=M3	lee M1 (6)	escribe M3 (6)
I5	100010100	Mover, orig.=M2, dest.=M4	lee M2 (4)	escribe M4 (4)
I6	011010000	Dec.-1, A=2	ALU: 2-1=1	escribe M1 (1)
I7	100001101	Mover, orig.=M1, dest.=M5	lee M1 (1)	escribe M5 (1)

TABLA IV. Simulación del ciclo Fetch-Decode-Execute-Memory/WB para las siete instrucciones codificadas.

C. Estado final de memoria

Al concluir la ejecución de las siete instrucciones, el contenido de las memorias coincide con el resultado esperado del algoritmo original (2+2=4, 5+1=6, M4=M2=4, 2-1=1):

M1 = 1	M2 = 4	M3 = 6	M4 = 4	M5 = 1
---------------	---------------	---------------	---------------	---------------

IV. CONCLUSIONES

El ejercicio permitió comprobar de forma práctica que una única línea de un algoritmo de alto nivel no siempre corresponde a una sola instrucción de máquina: las restricciones del conjunto de instrucciones —en este caso, un destino fijo para los resultados aritméticos— pueden obligar a descomponer una operación en varias instrucciones encadenadas. Simular el ciclo de instrucción etapa por etapa para cada una reforzó la diferencia, discutida previamente en clase, entre la ISA de un procesador (qué operaciones existen y cómo se codifican) y su implementación (cómo se ejecutan realmente esas etapas en el hardware). Este mismo razonamiento es la base conceptual sobre la que se construyen, más adelante en el curso, la programación en ensamblador real y el análisis de instrucciones de procesadores como el AVR y el MIPS.

FUENTE

Ejercicio en clase, nota “Ciclo de instrucción” (2025-08-25), apoyada en el repositorio del profesor Dr. Rubén Hernández Torres: github.com/rubenhIII/ArquitecturaComputadorasISC.

Desarrollo propio disponible en: github.com/davidelpollon-eng/arquitecturadecomputadoras.